

Exploring PPO Extensions for Lux AI Season 3: Integrating Planning Heuristics and Multi-Policy Training

Silja Stubhaug Torkildsen¹ and Karl-Henrik Horve¹

¹Master’s Student, Machine Learning, University of Bergen, Norway

April 27, 2025

Abstract

The Lux AI Season 3[2] challenge requires agents to explore a fog-of-war environment, collect energy, and control relics while competing against an opponent. In this project, agents based on Proximal Policy Optimization (PPO) were independently developed to explore different extensions inspired by recent research. One approach integrated dynamic re-planning using the D* Lite algorithm combined with multi-agent coordination heuristics derived from the MAPPOHR[3] framework. Another approach applied multi-agent and multi-policy techniques to solve the task. Due to implementation challenges, we were unable to directly compare the models performance against each other by making them play each other. Evaluation was performed using training metrics such as value loss, policy gradient loss, explained variance, and entropy loss, extracted during the learning process.

agents must collect energy and control relics while competing against an opponent. The map is randomly generated at the start of each game and remains the same across all five matches within the game. Agents operate under fog-of-war conditions, observing only a limited area surrounding their units at each timestep. Each unit can perform discrete actions, including moving in four directions, using a sap ability to interact with objects, or remaining stationary. Energy consumption occurs with movement. Points are earned by sapping and controlling relics, with the final score determining the winner after a fixed number of turns. Observations available to agents include local terrain information, visibility masks, nearby relics, unit energy levels, and the current game clock. Due to limited observations, agents must make decisions based on partial information, balancing exploration, energy management, and strategic positioning against opponents.

1 Introduction

The Lux AI Season 3 competition autonomous agents to operate in a partially observable, competitive environment. Each player controls multiple units and must explore the map, manage energy resources, and capture relics while facing an opponent team. Although the environment involves competition, the problem can be addressed using single-agent reinforcement learning methods or handcrafted strategies, without requiring centralized multi-agent learning. Previous research has explored combining classical planning algorithms such as A* or D* Lite[3] with reinforcement learning to improve sample efficiency and adaptability.

2 Experiments

2.1 Base Environment

The Lux AI Season 3 environment is a two text-dimensional grid world with fog-of-war, where

2.2 Multi -Policy and -Agent Solution

2.2.1 Motivation: Framework Flexibility for Custom Architectures

Addressing the Lux AI S3 challenge involved navigating a complex, multi-agent environment with a structured action space. This prompted exploration into advanced agent architectures and required a flexible implementation framework capable of handling potentially complex network designs and multi-agent training.

For instance, one architectural direction conceptually explored, inspired by the principle of component decomposition in models like that of [1], was to separate components within our own action space definition. The baseline Lux action input is (N,3) per unit (action type, dx, dy), but the dx/dy parameters are only used for the ‘sap’ action (type 5). We considered creating a dedicated network head to output dx/dy values, trained conditionally only when the ‘sap’ action was selected. The hypothesis was that such separation could potentially en-

hance learning efficiency and stability by focusing parameter updates and simplifying the exploration problem, ensuring gradients for these specific parameters only flow when relevant.

While this specific ‘separate heads’ architecture remained conceptual for this project and was not implemented in the agents whose results are presented below (see Appendix A for conceptual code snippets¹), the desire to accommodate such potentially complex, conditional network structures was a primary factor in our technical approach. This requirement, alongside the need for effective multi-agent training capabilities, led us to select the Ray RLlib framework for this line of investigation. RLlib offers strong support for custom models and multi-agent scenarios, providing the necessary flexibility for exploring these advanced techniques.

2.2.2 Environment structure

The RLlib framework allowed us to use GPU acceleration for training, more freedom in designing the neural networks, and strong support for multi-agent training and multi-policy implementations. To facilitate integration with both the LuxAI Season 3 engine and RLlib, we created the following layers of environments and wrappers:

1. **LuxAIS3Env** - The underlying environment the luxai_s3 engine uses (not created by us).
2. **LuxAIBase** - Our base environment that interacts with the luxai environment.
3. **FlattenNormalizeObservation** - A wrapper layer that normalizes and flattens the observations produced by LuxAIBase.
4. **FlattenActionWrapper** - Another wrapper layer that reconstructs the flattened action vector produced by the agent’s policy into the format expected by LuxAIBase.
5. **LuxMultiAgentEnv** - The last layer that facilitates multi-agent, multi-policy training.

A custom reward shaping mechanism was developed to provide denser feedback. Agents received scaled rewards (weight=0.1) for positive point changes per step, plus a match win bonus (1) awarded only at match conclusion if the win was by score. This aimed to balance immediate scoring with the ultimate goal of winning matches.

2.2.3 Neural network Architecture

This agent utilizes RLlib’s default PPO model structure. To allow the agent to remember intra-match temporal data such as relic and enemy locations, we augmented the model structure with an

LSTM layer to process sequential information. The LSTM cell was configured to 256 with a processing sequence of length 20.

2.2.4 Multi-Agent setup

The agent was configured to learn two independent policies, [p0, p1], one for each player, and trained simultaneously by our multi-agent environment setup.

The agent was trained with the following hyperparameters:

- Learning Rate: 0.00025
- Gamma(discount factor): 0.99
- Gradient Clipping: 0.5
- PPO Epochs: 5
- Minibatch Size: 512

The agent was trained with 8 runners running 4 environments each for a total of 32 parallel environments. Each iteration takes 505 step in each of the parallel environments, and over the 100 iterations this resulted in a total of 1 616 000 training steps.

2.2.5 Training Results and discussion

The mean rewards per agent (P0 and P1) show significant variance throughout the training process, with occasional large spikes in rewards, they do not exhibit a clear, consistent upward trend towards stabilization within the 100 iterations observed. The reward curves for both agents are largely overlapping, indicating similar reward acquisition patterns during training.

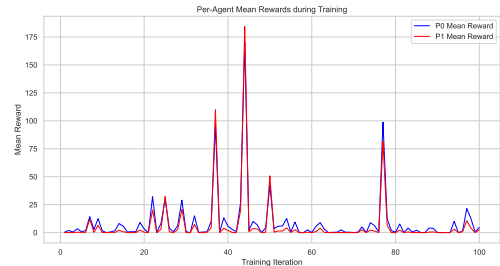


Figure 1: Multi-agent mean rewards.

The policy loss for both agents demonstrates considerable fluctuations. This suggests that the policy networks aren’t learning properly and that updates are unstable or that it is a complex learning task.

The value function loss exhibit large, intermittent spikes, particularly for agent P1 between iterations 50 and 75 where the loss appears capped at a value of 10. These spikes indicate periods where predicting the value of states was particularly challenging or unstable for the agents. Outside these spikes, the VF loss generally remains low.

¹or the git-repository for the full conceptual agent

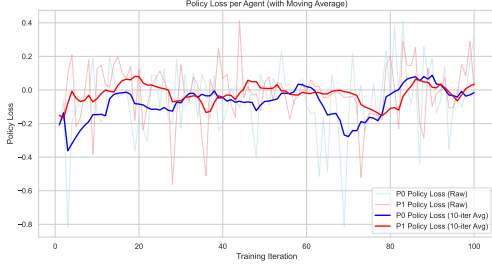


Figure 2: Multi-agent policy loss

At this point, it is also important to consider the potential impact of the training configuration on this observed instability. The training utilized 32 parallel environments, each initialized with the same random seed per training run. This means that within a collected batch of experiences, all environments would encounter identical pseudo-random events, such as map generation or relic spawns. Such correlation across the batch likely amplifies the variance in collected rewards – when a particularly favorable or unfavorable seed occurs, it affects all environments simultaneously, contributing to the large reward spikes observed. Furthermore, this lack of stochastic diversity within batches can destabilize the learning of the value function, as updates are based on highly correlated and potentially extreme outcomes, likely contributing to the observed spikes in VF loss.

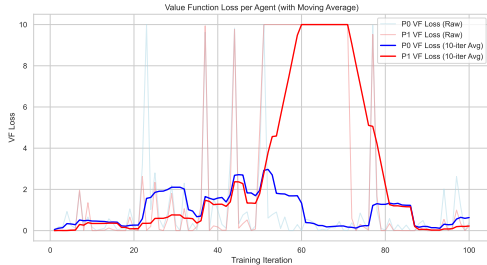


Figure 3: Multi-agent Value Function loss

Policy entropy, which measures the randomness or exploration in the agent’s policy, shows a divergence between the two agents. P0’s entropy fluctuates but remains relatively high, while P1’s entropy shows a more pronounced decreasing trend, dropping from around 68 to below 63 towards the end of training. This suggests that policy P1 became more deterministic in its actions over time compared to P0.

Regarding the magnitude of reward spikes approaching 175-200 mean reward per agent per episode in some iterations; We cannot entirely rule out implementation errors without further investigation, the very high total episode scores are plausible, especially considering the symmetry of the

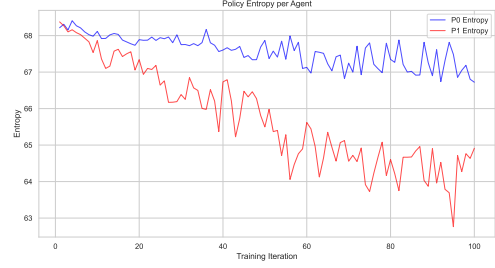


Figure 4: Multi-agent Entropy

spikes seeing as relics are placed simultaneously on each half of the map. In other words, these spikes could indicate very early and favorable relic spawns for both teams, leading to massive point gains across the episode.

To summarize; The current training results show that this solutions requires either tweaking for faster convergence, or many, many more training iterations before it can be considered a challenging opponent.

2.3 Dynamic Re-planning and MAPPOHR Coordination

This solution extends the Lux AI Season 3 environment with a modular wrapper stack, designed to integrate dynamic path planning, real-time re-planning, and heuristic coordination strategies. The wrapper stack includes:

- **SB3LuxEnvBase:** A basic Stable-Baselines3-compatible wrapper around Lux S3. It handles action translation, environment configuration, exploration tracking, and initial rule-based reward shaping.
- **SB3LuxEnvStaticPlanner:** Builds on the base wrapper by integrating static path planning via A* to guide unit movement efficiently across the map. It replans globally at configurable intervals and adds strategic context to certain actions, such as sap actions near relic nodes or strategic map locations.
- **SB3LuxEnvMAPPO:** Further extends the static planner with dynamic local re-planning using D* Lite, collision risk detection, and multi-agent heuristic coordination. Units dynamically adapt their paths when encountering obstacles or enemies, and coordinate to avoid traffic jams, deadlocks, and poor sap targeting.

2.3.1 Reward shaping

The reward structure is designed to promote balanced exploration, resource gathering, and tactical combat. Units are rewarded for discovering

new tiles, expanding overall map coverage, moving toward unexplored frontiers, and spreading out efficiently across the map. Additional incentives are given for finding and controlling relic nodes, maintaining long-term presence on point tiles, and positioning units strategically near relics. Movement rewards encourage units to travel away from their spawn point and maintain high energy levels, while combat rewards emphasize effective use of the sap action, especially when targeting multiple enemies or defending key areas. Penalties are applied for inefficient behaviors such as repeated re-visits to the same tiles, excessive waiting or re-planning, low-energy sap attempts, and poor collision management. As the match progresses, the reward focus dynamically shifts from exploration toward relic control, ensuring the agent adapts its priorities throughout the game. Extensions in the static and MAPPO wrappers further refine these incentives by encouraging smart territory control and coordinated multi-agent behavior.

2.3.2 Training configuration

Training was performed using the PPO implementation from Stable-Baselines3 [4], but with two distinct neural network configurations depending on the environment wrapper. For the Base and Static Planner wrappers, training used the standard `MultiInputPolicy` from Stable-Baselines3. The network architecture consisted of a fully connected multilayer perceptron (MLP) with hidden layers of sizes [512, 256, 128], using ReLU activation functions. In contrast, for the MAPPOHR wrapper, a custom recurrent architecture was developed. Training employed a specialized RNN policy (`RNNPolicy`) combined with a custom feature extractor (`LuxFeaturesExtractor`). The feature extractor processes spatial information (maps, sensor masks, energy fields) through convolutional layers and structured data (unit states, relics, game configuration) through dedicated multilayer perceptrons, ultimately merging all features into a unified representation. An LSTM module was appended after feature extraction to capture temporal dependencies crucial for partially observable environments. Separate two-layer MLPs were used for policy and value outputs, supporting recurrent policy optimization. Training across all variants used eight parallel environments with `SubprocVecEnv`, observation and reward normalization via `VecNormalize`, and a linearly decaying learning rate schedule. The PPO hyperparameters were consistent across setups: an initial learning rate of $5e-4$ decaying to $1e-4$, `n_steps` set to 2048, `batch_size` of 128, `n_epochs` set to 5 and discount factor `gamma` of 0.99.

2.4 Results

Training stability was assessed using entropy loss and value loss metrics.

2.4.1 Entropy Loss

Entropy loss measures the policy’s level of exploration during training. Figure 5 shows the entropy loss curves across 1 million time steps. All agents quickly reduced their entropy loss early in training, meaning they shifted from random exploration to more consistent decision-making. By the end of training, the entropy loss values became similar across all agents, showing that they reached comparable levels of confidence in their policies.

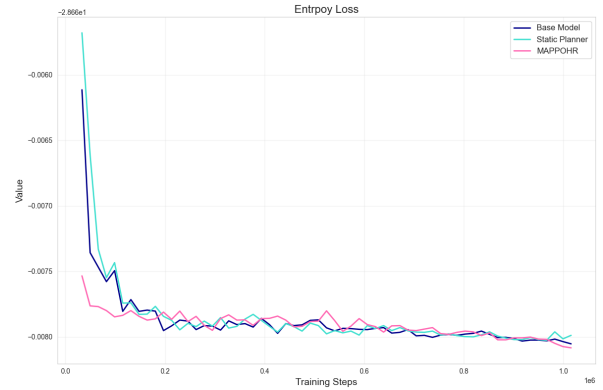


Figure 5: Entropy loss during training for baseline, static planner, and MAPPOHR agents.

2.4.2 Value Loss

Value loss reflects the accuracy of value function predictions. Figure 6 displays the value loss curves across training. The MAPPOHR agent exhibited higher variation and magnitude in value loss compared to the baseline and static planner agents. This suggests that dynamic re-planning introduced greater difficulty in learning stable value predictions.

2.4.3 Policy Gradient Loss

Policy gradient loss tracks how much the agent’s policy is updated during learning. Figure 7 shows the policy gradient loss curves across 1 million time steps. All agents exhibited an initial phase of strong policy updates, followed by stabilization near zero after approximately 200,000 steps. The MAPPOHR agent exhibited smaller policy updates from the beginning of training compared to the baseline and static planner agents, maintaining closer proximity to zero throughout training. This indicates more conservative and stable policy adaptation.

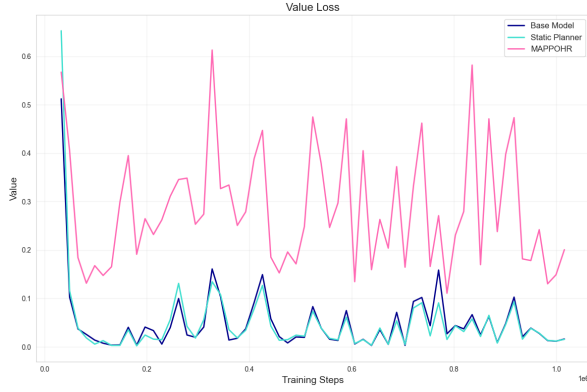


Figure 6: Value loss during training for baseline, static planner, and MAPPOHR agents.

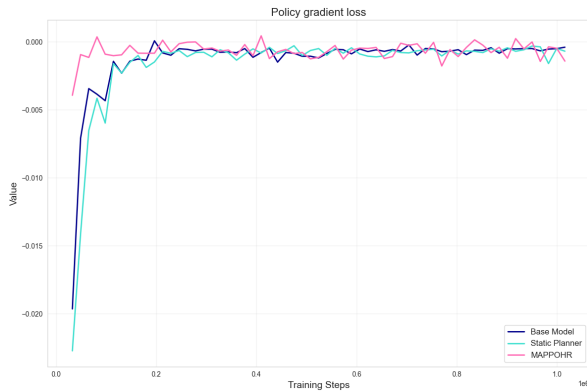


Figure 7: Policy gradient loss during training for baseline, static planner, and MAPPOHR agents.

2.4.4 Final Reward

Task performance was evaluated based on the final reward accumulated during training episodes. Figure 8 presents the mean final rewards. Overall, the final reward reflects the agent’s combined ability to explore efficiently, interact with the environment, and strategically control key objectives over the course of an episode. All agents demonstrated high variability in rewards, reflecting challenges inherent to exploration under partial observability. Despite this, the MAPPOHR agent achieved reward levels comparable to the baseline and static planner agents.

3 Conclusion

The RLLib-based multi-agent approach showed significant training instability. While reward spikes hinted at high scoring potential, key metrics like mean rewards, policy loss, and value function loss fluctuated without clear convergence within 100 iterations. Differing policy entropy between agents suggested varied exploration, but overall instability indicates the need for further tuning, extended

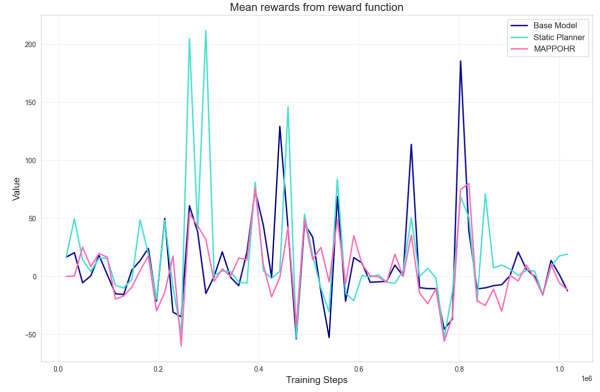


Figure 8: Final reward trends during training for baseline, static planner, and MAPPOHR agents.

training, or architectural changes.

Conversely, the Stable-Baselines3 agent integrating D* Lite and MAPPOHR heuristics demonstrated more stable learning dynamics compared to baseline agents. Although its dynamic re-planning increased value loss variance, policy updates were more conservative, and final rewards were comparable to simpler agents, indicating potential viability. The entropy loss for all SB3-based agents converged similarly, showing a healthy transition from exploration to exploitation.

A significant limitation was the inability to perform direct head-to-head comparisons between the RLLib and SB3/MAPPOHR agents due to implementation challenges. Consequently, assessments relied on individual training metrics (loss curves, rewards, entropy), which offer insights into learning stability but don’t definitively measure relative competitive performance.

To summarize, Lux AI S3 was challenging due to partial observability and multi-unit coordination. Our experimentation highlighted difficulties achieving stable convergence with complex RLLib setups, while integrating classical planning (D* Lite) and heuristics (MAPPOHR) with PPO showed promise in managing complexity, despite challenges in value prediction. Future work should focus on extended training, hyperparameter optimization, exploring architectural variations like the separate sap-head, and developing methods for direct agent evaluation.

4 Code Availability

The code that was used to implement the model and produce the results presented in this paper is hosted on GitHub [5].

A Code outtakes for Separate Sap-Target Head Concept

See the Git Repo for the full definition.

Figure 9 shows the proposed network architecture, and figure 10 shows the update method. Whitespace and docstrings have been removed to keep it within a page. For a full, more readable version, see the git repository.

References

- [1] Kalle Kujanpää, Willie Victor, and Alexander Ilin. “Automating Privilege Escalation with Deep Reinforcement Learning”. In: *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*. arXiv:2110.01362 [cs]. Nov. 2021, pp. 157–168. DOI: 10.1145/3474369.3486877. URL: <http://arxiv.org/abs/2110.01362> (visited on 04/27/2025).
- [2] *Lux-AI-Challenge/Lux-Design-S3*. original-date: 2024-08-18T01:26:23Z. Mar. 2025. URL: <https://github.com/Lux-AI-Challenge/Lux-Design-S3> (visited on 04/27/2025).
- [3] Shaoming Peng. *Multi-Robot Path Planning Combining Heuristics and Multi-Agent Reinforcement Learning*. arXiv:2306.01270 [cs]. June 2023. DOI: 10.48550/arXiv.2306.01270. URL: <http://arxiv.org/abs/2306.01270> (visited on 04/27/2025).
- [4] *PPO — Stable Baselines3 2.6.1a0 documentation*. URL: <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html> (visited on 04/27/2025).
- [5] Silja Stubhaug Torkildsen. *SiljaTorki/LUX_AI*. original-date: 2025-03-13T00:20:38Z. Apr. 2025. URL: https://github.com/SiljaTorki/LUX_AI (visited on 04/27/2025).

```

import torch
import torch.nn as nn
from torch.distributions import Categorical
class ActorCritic(nn.Module):
    def __init__(self, num_inputs, num_outputs_action_type, num_outputs_sap_target,
        ↪ hidden_dim=256):
        super(ActorCritic, self).__init__()
        # Shared layers
        self.shared_layers = nn.Sequential(
            nn.Linear(num_inputs, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )
        # Actor head: Action Type Logits
        self.action_type_head = nn.Linear(hidden_dim, num_outputs_action_type)
        # Actor head: Sap Target Coordinates (Direct Regression)
        self.sap_target_head = nn.Linear(hidden_dim, num_outputs_sap_target)
        # Critic head: Value Estimate
        self.value_head = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        shared_features = self.shared_layers(x)
        action_type_logits = self.action_type_head(shared_features)
        sap_target_coords = self.sap_target_head(shared_features) # Direct output
        value_estimate = self.value_head(shared_features)
        return action_type_logits, sap_target_coords, value_estimate

    def evaluate_actions(self, obs, action_types_flat, sap_targets_flat):
        action_type_logits_flat, current_sap_targets_flat, values = self.forward(obs)
        batch_size = obs.size(0)
        max_units = action_types_flat.size(0) // batch_size if batch_size > 0 else 0 #
        ↪ Handle empty batch case? No, PPO loop shouldn't have empty batches.
        num_action_types = action_type_logits_flat.size(-1) // max_units
        # Action Type Distribution - Reshape logits: (batch_size * max_units,
        ↪ num_action_types)
        dist = Categorical(logits=action_type_logits_flat.view(-1, num_action_types))
        new_log_probs_flat = dist.log_prob(action_types_flat) # Log prob of the
        ↪ actions *actually taken*
        entropy_flat = dist.entropy()
        # Reshape predicted sap targets to match the flat action structure
        current_sap_targets_flat = current_sap_targets_flat.view(-1, 2) # Shape:
        ↪ (batch_size * max_units, 2)
        return new_log_probs_flat, entropy_flat, values, current_sap_targets_flat

    def get_action_value(self, obs, action_type=None, sap_target=None):
        action_type_logits, sap_target_coords, value = self.forward(obs)
        # Action Type Distribution
        action_probs = Categorical(logits=action_type_logits)
        if action_type is None:
            # Sample action type if not provided
            action_type = action_probs.sample()
            # Calculate log probability and entropy for the action type
            action_type_log_prob = action_probs.log_prob(action_type)
            entropy = action_probs.entropy()
            if action_type is None: # If we were sampling
                return action_type, sap_target_coords, action_type_log_prob, entropy,
                ↪ value
            else: # If we were evaluating provided actions
                return action_type_log_prob, entropy, value
    def get_value(self, obs):
        _, _, value = self.forward(obs)
        return value

```



```

# Conditional Sap Target Loss (MSE for sap actions only)
sap_action_code = 5 # The integer code for the sap action
sap_mask = (act_types_batch_flat == sap_action_code).float()

# Calculate MSE loss between predicted and actual sap targets
# current_sap_targets_flat: (batch_size * max_units, 2) - Network's current
→ prediction
# sap_targets_batch_flat: (batch_size * max_units, 2) - Actual targets stored in
→ buffer
sap_target_loss_unmasked = F.mse_loss(current_sap_targets_flat,
→ sap_targets_batch_flat, reduction='none').mean(dim=1) # Mean over dx, dy -> Shape:
→ (batch_size * max_units,)

# Apply mask and calculate mean over *only* the sap actions that occurred
sap_target_loss = (sap_target_loss_unmasked * sap_mask).sum() / (sap_mask.sum() +
→ 1e-8) # Avoid division by zero

# Total Loss
loss = (pg_loss
      - ent_coef * entropy_loss
      + vf_coef * v_loss
      + sap_coef * sap_target_loss)

```

Figure 10: Outtake from the PPO update function